



Fabien Campillo

2025

Data Sciences and Spikes

Fabien Campillo

Sep 23, 2025

CONTENTS

I	Content	3
1	Resources and references	5
1.1	Some Python packages	5
1.2	Resources and references for general data sciences	6
1.3	Resources and references for data sciences in neurosciences	6
2	Electrophysiology Data	9
2.1	Types of Electrophysiology Recordings	9
2.2	Common Electrophysiology Data Formats	10
2.3	Most Used Electrophysiology Data Formats	10
2.4	Databases	13
2.5	Gateways with Python	15
3	PyABF	17
3.1	Using External Python Packages	17
3.2	pyABF on the net	17
3.3	Exploring abf files	18
3.4	Plots	22
II	Appendices	27
4	Installing Python and some tools	29
4.1	Main Python distributions for data sciences	29
4.2	Installing Anaconda and Miniconda	31
4.3	Conda	32
4.4	Conda virtual environment	33
4.5	Setting Up the Conda Virtual Environment for This Project	36
5	Interactive Computing with Jupyter Ecosystem	39
5.1	Jupyter and around	39
5.2	Colab	40
5.3	Jupyter {book}	40
6	Diagnostic	43
6.1	Myst	43
6.2	Emoji Test Page	43

This Jupyter Book is designed to help you get started in data science by exploring electrophysiological data, especially spike train recordings, in a practical and accessible way.

Even though we are mainly interested in processing electrophysiology measurements such as spikes, we will attempt an overview of neuroscience resources.

We will focus on electrophysiology data processing and distinguish between:

- M/EEG data, non-invasive/extracranial,
- and invasive data at the single neuron level or from a population of neurons, notably using MEA (Multi-Electrode Array).

It is this second category that is of most interest to us (MathNeuro). The first category is very well developed. Processing extracranial electrophysiological data (EEG/MEG) is generally more complex than processing intracranial measurements (spikes, LFP, ECoG). In intracranial recordings, electrodes are close to neurons: the signal is more localized, with a better signal-to-noise ratio, which facilitates the identification of action potentials or local fields. In contrast, extracranial signals are heavily attenuated, resulting from the summation of millions of neurons and distorted by cranial tissues. They are also contaminated by numerous artifacts. Analysis therefore requires advanced processing (filtering, correction, modeling) and solving the inverse problem (retrieving brain sources from incomplete and ambiguous measurements, which is a mathematically ill-posed problem).

Important

This notebook relies on Python packages such as `numpy`, `matplotlib`, `pyabf`, `seaborn`, and others. To ensure reproducibility and avoid conflicts with other Python projects, it is **strongly recommended to use a dedicated virtual environment**. For detailed instructions on setting up the environment, see the beginning of Chapter [PyABF](#).

This Jupyter Book is part of the Data Science Bootcamp for [MathNeuro](#) and is made openly accessible to the broader community.

This Jupyter book <https://fabien-campillo.github.io/data-science-spikes/> • The GitHub repository <https://github.com/fabien-campillo/data-science-spikes>

*Several parts of this book, including sections of the Markdown content and Python source code, were generated or refined with the assistance of **ChatGPT-4**, which also provided guidance on building this Jupyter Book. Some of the prompts used with ChatGPT are preserved as comments in the Markdown cells, providing a peek into the questions and guidance that shaped the content. While this tool was helpful in drafting and organizing content, all remaining errors and final decisions are entirely my own.*

By [Fabien Campillo](#) Email me © Copyright 2025. This work is licensed under CC BY-NC-SA 4.0 (Creative Commons Attribution-NonCommercial-ShareAlike).

Part I

Content

RESOURCES AND REFERENCES

I've put together some resources and references using Python (but keep in mind, R is another popular route into data science).

1.1 Some Python packages

First, I list some indispensable Python libraries used in data science. In addition to core Python, you should also start getting familiar with a few other tools:

- Python's classics:
 - [NumPy](#) – numerical computing and array manipulation.
 - [SciPy](#) – scientific computing and statistics.
 - [Matplotlib](#) – basic plotting library.
- Data Manipulation:
 - [Pandas](#) – data structures and analysis tools.
- Statistical Analysis:
 - [statsmodels](#) – estimation of statistical models, statistical tests, and data exploration.
- Machine Learning:
 - [Scikit-learn](#) – widely used machine learning library.
- Natural Language Processing (NLP):
 - [NLTK](#) – platform for working with human language data.
 - [SpaCy](#) – main library for NLP tasks.
 - [Gensim](#) – topic modeling library.
- Data Visualization:
 - [Seaborn](#) – statistical data visualization.
 - [Plotly](#) – interactive graphing library.
- Web Scraping:
 - [BeautifulSoup](#) – extracting data from HTML files.

1.2 Resources and references for general data sciences

1.2.1 Books

- **An Introduction to Statistical Learning with Applications in Python**, Springer 2023, by Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor. See [Book Homepage and Resources](#) with the [PDF](#) and the associated [Youtube videos](#) with Trevor Hastie & Jonathan Taylor (and it starts with Trevor complimenting Jonathan on his new haircut, why not...). [Trevor Hastie](#) is one of the big dudes in statistics (see his [book](#) “The Elements of Statistical Learning: Data Mining, Inference, and Prediction”), and [Jonathan Taylor](#) is a younger statistician with a nice new haircut.
- **Python for Data Analysis** (3rd ed.), O'Reilly 2022, by [Wes McKinney](#) a creator of Panda. The [open edition](#) is available, with the [codes](#), see his [GitHub](#) for other resources.
- **Python Data Science Handbook** (2nd ed.), O'Reilly 2022, by [Jake VanderPlas](#) – [full text](#), and the associated [Jupyter Notebook](#) (very nice!), see his [GitHub](#) for other resources.
- **Practical Statistics for Data Scientists: 50+ Essential Concepts Using R and Python** (2nd ed.), O'Reilly 2020, by Peter Bruce, Andrew Bruce, Peter Gedeck. See the [GitHub](#) with the [Python codes and notebooks](#).
- **Python for Probability, Statistics, and Machine Learning** (3rd ed.), Springer 2022, by José Unpingco. See his [GitHub](#) for other resources.
- **Hands-On Machine Learning with Scikit-Learn, Keras & Tensorflow** (3rd ed.), O'Reilly 2022, by [Aurélien Géron](#), and the associated [notebooks](#), see his [GitHub](#) for other resources. Machine learning and deep Learning.
- **Deep Learning Illustrated**, Addison-Wesley 2019, by [Jon Krohn](#), with the associated [notebooks](#), the concept if quite interesting. See also this [github](#).

I do not provide references on the basic mathematical foundations of data science, which usually include linear algebra, calculus (with a focus on optimization), probability theory, statistics (both elementary and inferential), discrete mathematics (graphs, combinatorics, logic), and sometimes numerical methods. I also do not include general references on statistics, machine learning, or Python programming itself, as well as topics related to databases such as SQL, relational database design, and NoSQL systems. There are numerous high-quality resources available for all these areas.

1.2.2 Jupyter (note)books

Among the previous references:

- [Jake VanderPlas' Python Data Science Handbook](#)
- [Aurélien Géron's notebooks](#), *a series of Jupyter notebooks that walk you through the fundamentals of Machine Learning and Deep Learning in Python using Scikit-Learn, Keras and TensorFlow 2.*
- [Wes McKinney's "Python for Data Analysis" open edition and notebooks.](#)

1.3 Resources and references for data sciences in neurosciences

1.3.1 References

- **Python in Neuroscience**, E. Muller, J. A. Bednar, M. Diesmann, M.-O. Gewaltig, M. Hines, and A. P. Davison *Frontiers in Neuroinformatics*, 9, 2015.
- **Case Studies in Neural Data Analysis**, 2016 - The book presents MATLAB tools, but there is an associated [GitHub repository](#) for Python. The book primarily covers extracranial data, except Chapter 8: *Basic Visualizations and Descriptive Statistics of SpikeTrainData*.

- **Neural Data Science** (2020–23), Aaron J. Newman from the NeuroCognitive Imaging Lab (Dalhousie University, Halifax).
Starts from scratch, especially in Python. Includes a section on [Single Unit Data](#). See the [GitHub repository](#) for the Jupyter Book and the YouTube channel [Neural Data Science with Python](#).
- **Neural Data Science: A Primer with MATLAB and Python**, Erik Lee Nylen and Pascal Wallisch, 2017.
See the [table of contents](#).

1.3.2 Spike Train and Electrophysiology Data Analysis

Python packages

- **syncopy** - Systems Neuroscience Computing in Python: a Python package for large-scale analysis of electrophysiological data, with the following [article](#).
- **MNE** - Open-source Python package for exploring, visualizing, and analyzing human neurophysiological data: MEG, EEG, sEEG, ECoG, NIRS, and more).
- **pynapple** – Python Neural Analysis Package. Pynapple is a lightweight Python library for neurophysiological data analysis. See the article: [Pynapple, a toolbox for data analysis in neuroscience](#), 2023.
- **osl-ephys** - This package contains models for analysing electrophysiology data. It builds on top of the widely used MNE-Python package and contains analysis tools for M/EEG sensor and source space analysis. From the [Oxford Centre for Human Brain Activity Analysis Group](#), with this [GitHub repository](#) and this 2025 paper: [osl-ephys: a Python toolbox for the analysis of electrophysiology data](#).
- **NeuralEnsemble** – a community-based initiative to promote and coordinate open-source software development in neuroscience. **Inactive since 2022**.

Jupyter (note)book(s)

- **Spike sorting the ‘Do It Yourself’ way** a Jupyter book by [Christophe Pouzat](#) with the [gitlab repository](#). See also the [Probabilistic Spiking Neuronal Nets: Companion](#) associated with the book [Probabilistic Spiking Neuronal Nets](#) co-authored with Antonio Galves and Eva Löcherbach.

1.3.3 Blog(s) and blog posts.

- **Spikes and Bursts** — an interesting blog by [David Cabrera-Garcia](#), where he explores various [concepts](#). He also runs a [YouTube channel](#) and shares projects on [GitHub](#). An interesting post:
 - [Patch-clamp data analysis in Python: animate time series data](#).
- [Patch clamp electrophysiology analysis with Python \(2023\)](#) by [Vincenzo Mastrolia](#)

1.3.4 Misc.

- **ElecFeX** - A MATLAB-based Electrophysiological Feature eXtraction toolbox for single-cell intracellular recordings. See the article: [ElecFeX is a user-friendly toolbox for efficient feature extraction from single-cell electrophysiological recordings](#)

1.3.5 Other tools

Before analyzing data, we first need to read electrophysiology recordings and handle the different standards used.

- The [pyABF](#) library was created by [Scott Harden](#). We will return to that package in a future section.

ELECTROPHYSIOLOGY DATA

Accessing electrophysiology records can be difficult, and working with them is often cumbersome, as they typically require specific formats to be quickly accessible and usable. To make data more widely available, it is crucial to develop not only open-access databases but also standardized file formats. Historically, data formats were tied to the devices that generated them—often proprietary and incompatible with other systems. This fragmentation soon became a barrier to scientific progress. In what follows, we first introduce common file formats, then present several relevant databases, and finally show how to work with them in Python.

2.1 Types of Electrophysiology Recordings

Electrophysiology covers a wide range of recording techniques, each suited to different biological questions. Here are the main categories:

- **Intracellular Recordings** - *Sharp electrode recordings*: measure the membrane potential inside a single cell • *Whole-cell patch clamp*: provides detailed access to ionic currents, membrane potential, and synaptic inputs • *Single-channel recordings*: resolve the activity of individual ion channels.
- **Extracellular Recordings** - *Single-unit recordings*: detect action potentials (“spikes”) from individual neurons using fine electrodes • *Multi-unit recordings*: capture spikes from small groups of neurons near the electrode tip • *Multi-electrode arrays (MEA)*: record from dozens to thousands of electrodes simultaneously across a neural population.
- **Field Potential Recordings** - *Local field potentials (LFPs)*: measure summed synaptic activity and slower fluctuations in a local region • *ECoG (electrocorticography)*: records field potentials directly from the cortical surface.
- **Non-Invasive Recordings** - *EEG (electroencephalography)*: scalp recordings of brain activity with high temporal resolution • *MEG (magnetoencephalography)*: detects magnetic fields generated by neuronal currents.
- **Other Specialized Methods** - *EMG (electromyography)*: records muscle activity • *ERG (electroretinography)*: records retinal responses to light • *Patch-clamp in slices/in vivo*: advanced combinations allowing intracellular access in complex preparations.

2.2 Common Electrophysiology Data Formats

Format	Full Name	Typical Use	Openness	Notes
ABF	Axon Binary File (Molecular Devices)	Patch-clamp and intracellular recordings (pCLAMP, Clampex)	Proprietary	Very common in cellular electrophysiology; ABF1 (older), ABF2 (newer).
IBW	Igor Binary Wave (Igor Pro)	Waveform storage and analysis	Proprietary	Widely used in physiology labs with Igor Pro; supports multidimensional data.
HEKA^A	Patchmaster (DAT/PGF/PUF)	Patch-clamp recordings (HEKA amplifiers)	Proprietary	Popular in Europe; rich metadata but tied to vendor software.
CED SON	Cambridge Electronic Design (Spike2)	Extracellular recordings, spike trains	Proprietary	Common for multi-electrode recordings and stimulus protocols.
WCP	WinWCP File (Strathclyde)	Patch-clamp and voltage-clamp recordings	Semi-open	Free Windows-only software; popular in teaching and some labs; less standardized than ABF/HEKA.
NWB	Neurodata Without Borders	Sharing neurophysiology data (spikes, LFPs, imaging)	Open standard	Community-driven, based on HDF5; widely promoted for reproducibility and FAIR data.
BIDS-iEEG	Brain Imaging Data Structure (intracranial EEG extension)	EEG, ECoG, iEEG	Open standard	Growing adoption; integrates with neuroimaging standards.
MAT	MATLAB File (.mat)	Custom lab storage/analysis	Semi-open	Extremely common, but not standardized for sharing.
HDF5	Hierarchical Data Format 5	Large, structured datasets	Open	Flexible backbone format; NWB is built on HDF5.
EDF/I	European Data Format / BioSemi Data Format	EEG, PSG, clinical neurophysiology	Open	Standard in sleep studies and EEG; supported by many clinical systems.
CSV/I	Comma-/Tab-separated Values	Generic export of time series or metadata	Open	Universally readable, but loses rich metadata and structure.
WAV	Waveform Audio File (Windows)	Audio-like exports of signals/spike trains	Open	Native Windows format; sometimes used for stimulus or simplified data export.

2.3 Most Used Electrophysiology Data Formats

- **ABF (Axon Binary File)** - ABF is a proprietary binary format developed by **Molecular Devices**. It is widely used for **patch-clamp and intracellular recordings**, especially in cellular electrophysiology research. ABF files store both the raw electrophysiological signals and metadata, such as sampling rate, stimulus protocols, and experimental parameters. Versions include ABF1 (older) and ABF2 (newer).
- **NWB (Neurodata Without Borders)** - NWB is an **open, community-driven standard** designed for **sharing and long-term storage of neurophysiology data**. It is based on HDF5 and supports raw signals, metadata, experimental design, stimuli, and derived data. NWB promotes **reproducibility and FAIR principles**, and is increasingly adopted by labs worldwide.
- **IBW (Igor Binary Wave)** - IBW is the binary wave format used by **Igor Pro** software. It is popular for **waveform storage and analysis**, particularly in physiology labs. IBW supports multi-dimensional data, annotations, and is often part of custom analysis pipelines. Despite being proprietary, it remains widely used because of historical and workflow reasons.

2.3.1 Summary Table

Format	Developer	Proprietary / Open	Free to use?	Main Use	Adoption
ABF (Axon Binary File)	Originally Axon Instruments → now Molecular Devices	Proprietary (specs partially available, but tied to pCLAMP)	Reading libraries are free (e.g., pyABF), but creating/using ABF files requires pCLAMP , which is commercial	Electrophysiology (patch-clamp, voltage/current recordings)	Widely used in labs with Molecular Devices equipment
NWB (Neurodata Without Borders)	Community-driven (Allen Institute, HHMI Janelia, Berkeley Lab, INCF, etc.)	Fully open-source and community standard	Free (developed and maintained openly on GitHub)	Standardized storage/sharing of neuroscience data (e-phys, imaging, behavioral, meta-data)	Growing adoption in large-scale projects, especially for data sharing and FAIR science
IBW (Igor Binary Wave)	WaveMetrics, Inc.	Proprietary (closed format, documentation partly available)	Requires Igor Pro (commercial software). Some third-party libraries exist to read IBW for free	General scientific data analysis and visualization	Popular in biophysics, neuroscience, spectroscopy

2.3.2 Companies and formats

Company	HQ	Main Products (Neuronal EP)	Native Data Formats	Conversion / Notes
Molecular Devices (Axon Instruments)	USA	Patch-clamp amplifiers (Axopatch, Multi-Clamp), digitizers (Digidata), pCLAMP software	ABF (Axon Binary File): ABF1 (legacy), ABF2 (current)	Widely used in patch-clamp. Readable with AxoGraph, Clampfit. Python: pyABF, Neo. Export to NWB possible.
HEKA Elektronik (Harvard Bioscience)	Germany	Patch-clamp amplifiers (EPC series), Patchmaster software	.dat / .pgf / .pul	Proprietary binary; support in Neo . Conversion pipelines exist for NWB/NIX .
Multi Channel Systems (MCS, Harvard Bioscience)	Germany	Multi-electrode arrays (MEAs), in vitro & in vivo systems	.mcd (proprietary), .h5 (newer systems)	SDK/API for reading .mcd . .h5 is HDF5 and integrates more easily. Neo + NWB supported.
Blackrock Neurotech	USA	Utah arrays, high-density neural recording for BCI	NSx (continuous), NEV (spike/event)	Openly documented. MATLAB, Python APIs. Supported in Neo . Standard in BCI research.
Neuralynx	USA	In vivo extracellular recording systems	NCS, NSE, NEV, etc.	ASCII headers + binary. Readers available (MATLAB, Python). Supported in Neo , can export to NWB.
Ripple Neuro	USA	High-channel neural recording & stimulation systems (BCI, clinical)	Similar to Blackrock: NSx / NEV	MATLAB/Python SDK. Actively supports NWB .
Tucker-Davis Technologies (TDT)	USA	High-throughput recording, optogenetics, stimulation	.tsq / .tev / .sev	Proprietary binary. TDT SDK + Neo support. NWB export possible.
Intan Technologies	USA	Low-power amplifier chips & headstages (RHD, RHS series)	.rhd / .rhs	Binary with header metadata. Intan provides readers (C++, MATLAB, Python). Neo + NWB supported. Widely used in open-source rigs.
ADInstruments	New Zealand	PowerLab DAQ, LabChart software, teaching/research physiology tools	.adicht (LabChart files)	Proprietary but supported by LabChart & APIs. Neo support available. Some pipelines to NWB.
Kerr Scientific Instruments	New Zealand	In vitro slice & tissue electrophysiology rigs	Exports via DAQ hardware (often LabChart, NI, etc.)	Less standardized — depends on DAQ choice (often integrates with ADInstruments).

2.4 Databases

By the way, data really matter! You can look at the Wikipedia [list of neuroscience databases](#). We will consider four of them.

2.4.1 NLM Dataset Catalog

[NLM Dataset Catalog](#) is a catalog of biomedical datasets from various repositories, NIH National Library of Medicine (NIH/NLM).

- The NLM Dataset Catalog is essentially a registry pointing to datasets, not a direct host.
- You'll usually get redirected to the actual host (sometimes PhysioNet, OpenNeuro, or institutional repositories).
- If the dataset is downloadable via URL, you can use `requests` or `wget` in Python to fetch the `.abf` files, then open with `pyabf`.
- No standard Python API for the catalog itself, but you can scrape metadata via their JSON endpoints (if provided per dataset).

2.4.2 Dryad

[Dryad](#) (Data Archive for Neuroscience Discovery) is an open data publishing platform and a community committed to the open availability and routine re-use of all research data.

- Dryad datasets are hosted at datadryad.org.
- They expose a REST API (<https://datadryad.org/api/v2/>).
- You can query a DOI, get file download links, and then download with `requests`.

Examples of data sets:

- a [dataset](#) presenting electrophysiological properties from whole-cell patch clamped nucleus accumbens core medium spiny neurons from male rats and female rats recorded in different estrous cycle phases.
- a [dataset](#) containing whole-cell electrophysiological recordings (patch-clamp recordings) from three cell types in mice.

Example of usage:

```
import requests
import pyabf

doi = "10.5061/dryad.xxxxx" # replace with dataset DOI
r = requests.get(f"https://datadryad.org/api/v2/datasets/{doi}")
files = r.json()["included"]
for f in files:
    if f["attributes"]["filename"].endswith(".abf"):
        url = f["attributes"]["downloadUrl"]
        abf_data = requests.get(url)
        with open(f["attributes"]["filename"], "wb") as fh:
            fh.write(abf_data.content)
        abf = pyabf.ABF(f["attributes"]["filename"])
```

2.4.3 Dandi Archive

[Dandi Archive](#) (Distributed Archive for Neuroscience Data Integration) is the BRAIN Initiative archive for publishing and sharing neurophysiology data including electrophysiology, optophysiology, and behavioral time-series, and images from immunostaining experiments.

- DANDI hosts primarily neurophysiology data in NWB (Neurodata Without Borders) format ([link](#), not ABF).
- They have a Python client: `dandi` ([link](#)).

An example of data set:

- [Patch-seq recordings from mouse visual cortex](#). Whole-cell Patch-seq recordings from neurons of the mouse visual cortex from the Allen Institute for Brain Science, released in June 2020. The majority of cells in this dataset are GABAergic interneurons, but there are also a small number of glutamatergic neurons from layer 2/3 of the mouse visual cortex.

Example of usage:

```
pip install dandi
dandi download DANDI:000003 # replace with dataset identifier
```

If the dataset includes `.abf` files (rare, usually NWB), you can load them directly with `pyabf`. More commonly, you'd work with NWB using `pynwb`, not `pyabf`.

2.4.4 Zenodo

[zenodo.org](#) (named after Zenodotus, the first librarian of Alexandria) is an open-access research data repository built and hosted by CERN (the European Organization for Nuclear Research), with support from the European Commission.

- Zenodo provides a REST API: <https://zenodo.org/api/>.
- Each record has a DOI and you can list associated files.

Example of data sets:

- a [data set](#) of CA1 pyramidal cell recordings using an intact whole hippocampus preparation, including recordings of rebound firing (V2).

Example:

```
import requests
import pyabf

record_id = "1234567" # Zenodo record ID
r = requests.get(f"https://zenodo.org/api/records/{record_id}")
for f in r.json()["files"]:
    if f["key"].endswith(".abf"):
        url = f["links"]["self"]
        abf_data = requests.get(url)
        with open(f["key"], "wb") as fh:
            fh.write(abf_data.content)
        abf = pyabf.ABF(f["key"])
```

2.5 Gateways with Python

Formats commonly used in electrophysiology (ABF, NBW, IBW) are not always straightforward to handle. Fortunately, several Python libraries act as gateways to read and convert:

- **PyNWB** is a Python package for working with NWB files ([doc github](#)).
- **pyabf** is a Python library for reading electrophysiology data from Axon Binary Format (ABF) files ([github](#)), see also [abf explorer](#), a simple graphical application for quickly viewing axon binary format (ABF).
- IBW (Igor Binary Wave) is a Python parser for IBW (.ibw) and Packed Experiment (.pxp) files written by WaveMetrics' IGOR Pro software (see [igor2](#)). IBW is a bit less active in Python.

As we already had a quick peep at above, several Python tools are available for working with records from these databases, see Chapter [PyABF](#) for more details.

3.1 Using External Python Packages

We assume that the correct Python environment is already set up; see Section “*Setting Up the Conda Virtual Environment for This Project*” for details.

Let’s import the external libraries needed to work with the notebook:

```
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
import pyabf

print(f"numpy: {np.__version__}")
print(f"matplotlib: {matplotlib.__version__}")
print(f"seaborn: {sns.__version__}")
print(f"pyabf: {pyabf.__version__}")
```

```
numpy: 1.26.4
matplotlib: 3.8.4
seaborn: 0.12.2
pyabf: 2.3.8
```

3.2 pyABF on the net

The [pyABF](#) library was created by [Scott Harden](#). Scott Harden has made pyABF available as an open-source library, aiming to simplify the process of working with ABF files in Python, making it easier for researchers to analyze and visualize their data. You can find more about pyABF and its documentation on his website:

- a (good) [tutorial](#) by [Scott W Harden]
- [pyABF](#) - A simple Python interface for Axon Binary Format ABF files, with [git repository](#)
- in Python Package Index [pypi](#)

3.3 Exploring abf files

This first notebook aims to demonstrate how to analyze electrophysiological recordings from a single cell by:

- Extracting basic characteristics of the recorded signals.
- Plotting, for each recording trial (sweep, see below), the evolution of the membrane potential (voltage) and, if applicable, the injected current.

We consider abf files contained in the data directory `data/data_patch_clamp_bcam/records`:

```
from pathlib import Path
records_dir = Path("data/data_patch_clamp_bcam/records")
abf_files = list(records_dir.rglob("*.abf")) # recursive glob
print("the directory ", records_dir, " contains ", len(abf_files), " abf files")
```

```
the directory  data/data_patch_clamp_bcam/records  contains  63  abf files
```

We focus on specific records:

```
file_path = "data/data_patch_clamp_bcam/records/2024_06/06.26/C2/2024_06_26_0028.
↳abf"
```

3.3.1 abf files and pyabf library

ABF File Overview

An ABF (Axon Binary Format) file is a proprietary file format developed by Axon Instruments (now part of Molecular Devices) to store electrophysiological data from experiments. ABF files are commonly used to save data from experiments like patch-clamp recordings, where researchers measure electrical signals from biological systems (such as neurons or muscle cells). These files can store a variety of information, including:

- Data Traces: Time series data for one or more channels, representing signals such as voltage or current.
- Metadata: Information about the experiment, including settings for the recording, such as sampling rate, experiment type, and device configuration.
- Multiple Sweeps: An ABF file can contain multiple sweeps (individual trials or experimental runs), which may differ in parameters or conditions.

The ABF format is binary, making it efficient for large datasets, but it is not easily readable without specialized software or libraries.

3.3.2 pyabf Library

The `pyabf` library is a Python package designed to facilitate working with ABF files. It provides an easy-to-use interface to read and manipulate data stored in ABF files. The library makes it simpler for researchers to extract relevant information from ABF files, without having to manually parse the binary data.

Key Features of `pyabf`:

1. Load ABF Files: Load an ABF file into memory and provide access to its data.
2. Access Data Traces: Extract time-series data, such as voltage and current traces (from ADC channels).
3. Multiple Sweep Support: Handle multiple sweeps (individual experimental runs) within a single ABF file.
4. Extract Metadata: Retrieve metadata like channel names, experiment parameters, and other settings.
5. Sweep Navigation: Select and navigate through multiple sweeps (trials) and analyze their data individually.

Common Functions in `pyabf`:

- `ABF(file_path)`: Initializes an ABF object from a given file path, loading the data into memory.
- `setSweep(sweep_index)`: Selects a specific sweep (experimental run) by its index.
- `sweepY`: Extracts the voltage (or other signal) data for the current sweep.
- `sweepX`: Extracts the time vector for the current sweep.
- `sweepC`: Extracts the command input (if available) for the current sweep.
- `adcNames`: List of ADC channel names.
- `dacNames`: List of DAC channel names.

3.3.3 Basics about abf objects

Where we import the `pyabf` package and load a record file:

```
abf = pyabf.ABF(file_path)           # we load it
print(abf)                           # record characteristics
```

```
ABF (v2.9) with 2 channels (mV, pA), sampled at 10.0 kHz, containing 14 sweeps,
↳ having no tags, with a total length of 2.35 minutes, recorded with protocol
↳ "IV-DG".
```

Here `abf = pyabf.ABF(file_path)` creates an `abf` object that have:

- **attributes**: data stored in the object, and
- **methods**: functions that belong to an object and can be called to perform actions

Attributes

We can print more attributes:

```
print(f"{'File Path:':>20} {abf.abfFilePath}")
print(f"{'File Version:':>20} {abf.abfVersionString}")
print(f"{'Sampling Rate:':>20} {abf.dataRate} Hz")
print(f"{'Total Sweeps:':>20} {abf.sweepCount}")
print(f"{'ADC Channels:':>20} {abf.adcNames}")
print(f"{'DAC Channels:':>20} {abf.dacNames}")
print(f"{'Channel Units:':>20} {abf.sweepUnitsY}")
print(f"{'Experiment Date:':>20} {abf.abfDateTime}")
```

```
File Path: /Users/campillo/Documents/0-git.nosync/data-science-spikes/
↳ data/data_patch_clamp_bcam/records/2024_06/06.26/C2/2024_06_26_0028.abf
File Version: 2.9.0.0
Sampling Rate: 10000 Hz
Total Sweeps: 14
ADC Channels: ['IN 0', 'IN 1']
DAC Channels: ['OUT 0', 'OUT 1']
Channel Units: mV
Experiment Date: 2024-06-26 17:23:16.253000
```

Methods

You can list all the methods of an `abf` object with `print(abf.__dict__)`.

```
methods = [method for method in dir(abf) if callable(getattr(abf, method)) and
↳ not method.startswith("__")]
print("\n".join(methods))
```

```
_dtype
_getAdcNameAndUnits
_getDacNameAndUnits
_id_helper
_loadAndScaleData
_makeAdditionalVariables
_readHeadersV1
_readHeadersV2
getAllXs
getAllYs
headerLaunch
launchInClampFit
saveABF1
setSweep
sweepD
```

You have private methods (Prefixed with `_`), and:

- `getAllXs()`: Returns all time points (X-values) for every sweep, useful for plotting.
- `getAllYs()`: Returns all recorded signal values (Y-values) for every sweep.
- `headerLaunch()`: Likely a utility function for debugging or inspecting header information.
- `launchInClampFit()`: Opens the ABF file in ClampFit, a software from Molecular Devices used for electrophysiology data analysis.
- `saveABF1()`: Converts and saves the ABF file in version 1 format, which is older but sometimes required for compatibility.
- `setSweep(sweepIndex)`: Sets the current sweep (i.e., trial or recording segment) to a given index for further processing.
- `sweepD`: Likely an attribute or method that provides the time duration of a sweep.

Of course the main parts of the sweep are the **recorded signal** and the **command input**:

```
# Print voltage trace (recorded signal)
print(f"{'Voltage Trace (mV):':>25} {abf.sweepY}")

# Print command input (if available)
print(f"{'Command Input (mV)':>25} {abf.sweepC}")
```

```
    Voltage Trace (mV): [-65.4175 -65.4175 -65.4236 ... -64.6851 -64.6851 -64.
↳ 6851]
    Command Input (mV): [0. 0. 0. ... 0. 0. 0.]
```

3.3.4 Basic abf file exploration

Main abf attributes and methods

The abf object contains various attributes and methods that allow you to access metadata and data from the .abf file. Here are some useful attributes and how to call them:

```
print("List of sweep indexes:", ", ".join(map(str, abf.sweepList)))
```

```
List of sweep indexes: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13
```

```
sweep_index = 0 # Choose a specific sweep (e.g., first sweep -> index 0)
abf.setSweep(sweep_index)

print(f"{'Voltage Trace (mV)':>25} {abf.sweepY}") # Print voltage trace
↳(recorded signal)
print(f"{'Command Input (mV)':>25} {abf.sweepC}") # Print command input (if
↳available)
print(f"{'Recorded Channels':>25} {abf.adcNames}") # Check all available ADC
↳channels (recorded signals)
print(f"{'Command Channels':>25} {abf.dacNames}") # Check DAC channels (command
↳input signals)
```

```
Voltage Trace (mV): [-65.4175 -65.4175 -65.4236 ... -64.6851 -64.6851 -64.
↳6851]
Command Input (mV): [0. 0. 0. ... 0. 0. 0.]
Recorded Channels: ['IN 0', 'IN 1']
Command Channels: ['OUT 0', 'OUT 1']
```

Some statistics of 1 sweep

Here, we have selected `sweep_index = 0`, representing the first sweep in the ABF file. We then compute some basic statistics of the corresponding voltage trace, such as the mean, median, min, max, standard deviation, and range of the signal:

```
data = abf.sweepY # The voltage trace for the specific swweep

stats = {
    "Mean (mV)": np.mean(data),
    "Median (mV)": np.median(data),
    "Min (mV)": np.min(data),
    "Max (mV)": np.max(data),
    "Std Dev (mV)": np.std(data),
    "Range (mV)": np.ptp(data), # Max - Min
}

print("\nVoltage Trace Statistics:")
for key, value in stats.items():
    print(f"{key:>20}: {value:.3f}")
```

```
Voltage Trace Statistics:
      Mean (mV): -81.502
     Median (mV): -91.797
        Min (mV): -93.097
        Max (mV): -64.545
    Std Dev (mV): 12.916
       Range (mV): 28.552
```

3.4 Plots

3.4.1 Plotting the voltage trace distribution

```
%config InlineBackend.figure_format = 'retina'
```

this configuration, known as the inline backend, helps achieve a balance between good visual quality and manageable file size, see Section *Jupyter backends*.

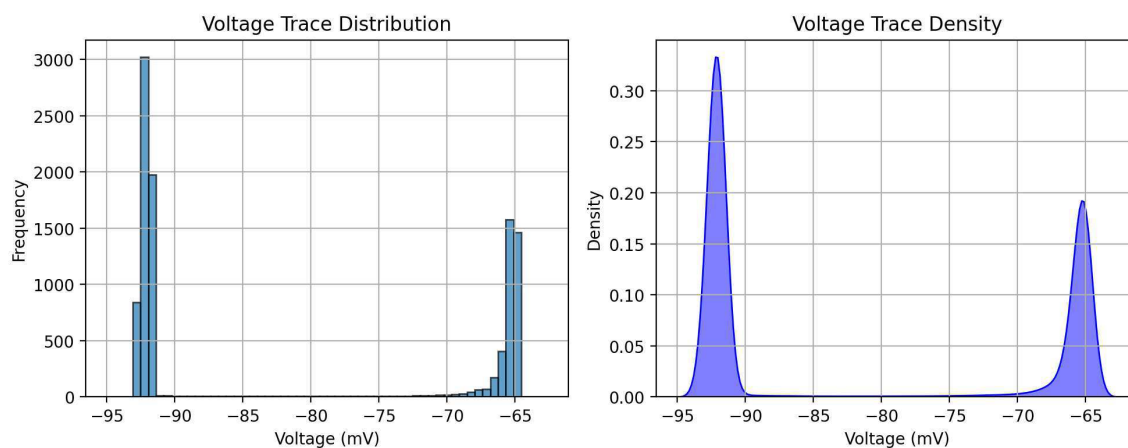
```
# to avoid warning from Seaborn internally calling a Pandas option
# (mode.use_inf_as_na) that has been deprecated in pandas ≥ 2.1.
import warnings # needed for filterwarnings
warnings.filterwarnings("ignore", category=FutureWarning, module="seaborn")

# Create a figure with two subplots (1 row, 2 columns), sharing x-axis
fig, axes = plt.subplots(1, 2, figsize=(10, 4), sharex=True)

# Plot histogram on the first subplot
axes[0].hist(data, bins=50, edgecolor='black', alpha=0.7)
axes[0].set_xlabel("Voltage (mV)")
axes[0].set_ylabel("Frequency")
axes[0].set_title("Voltage Trace Distribution")
axes[0].grid(True)

# Plot KDE on the second subplot
sns.kdeplot(data, bw_adjust=0.3, fill=True, color="b", alpha=0.5, ax=axes[1])
axes[1].set_xlabel("Voltage (mV)")
axes[1].set_ylabel("Density")
axes[1].set_title("Voltage Trace Density")
axes[1].grid(True)

# Adjust layout
plt.tight_layout()
plt.show()
```



3.4.2 Plotting one sweep with input current

```

color_adc = "C0"
color_dac = "C3"
my_lw = 0.8

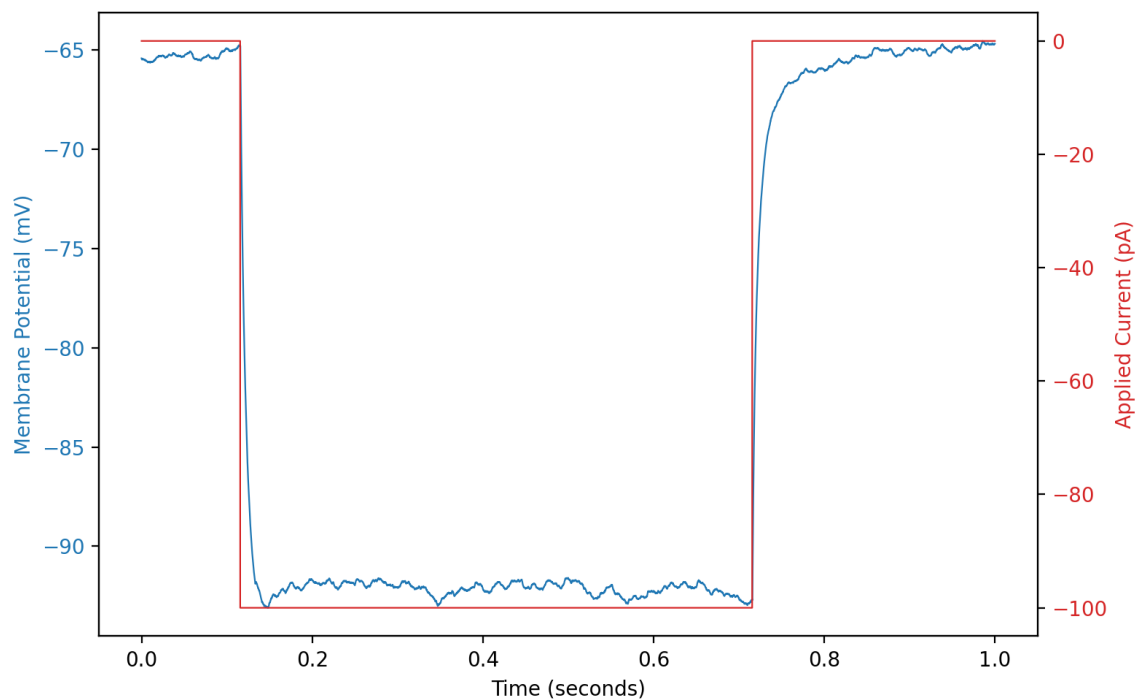
# Create the figure
fig, ax1 = plt.subplots(figsize=(8, 5))

# Plot the recorded curve (ADC) on the left axis
ax1.plot(abf.sweepX, abf.sweepY, color=color_adc, lw=my_lw, label="ADC waveform")
ax1.set_xlabel(abf.sweepLabelX)
ax1.set_ylabel(abf.sweepLabelY, color=color_adc)
ax1.tick_params(axis='y', labelcolor=color_adc)

# Create a second y-axis for the control curve (DAC)
ax2 = ax1.twinx()
ax2.plot(abf.sweepX, abf.sweepC, color=color_dac, lw=my_lw, label="DAC waveform")
ax2.set_ylabel(abf.sweepLabelC, color=color_dac)
ax2.tick_params(axis='y', labelcolor=color_dac)

# Improve the layout
fig.tight_layout()
plt.show()

```

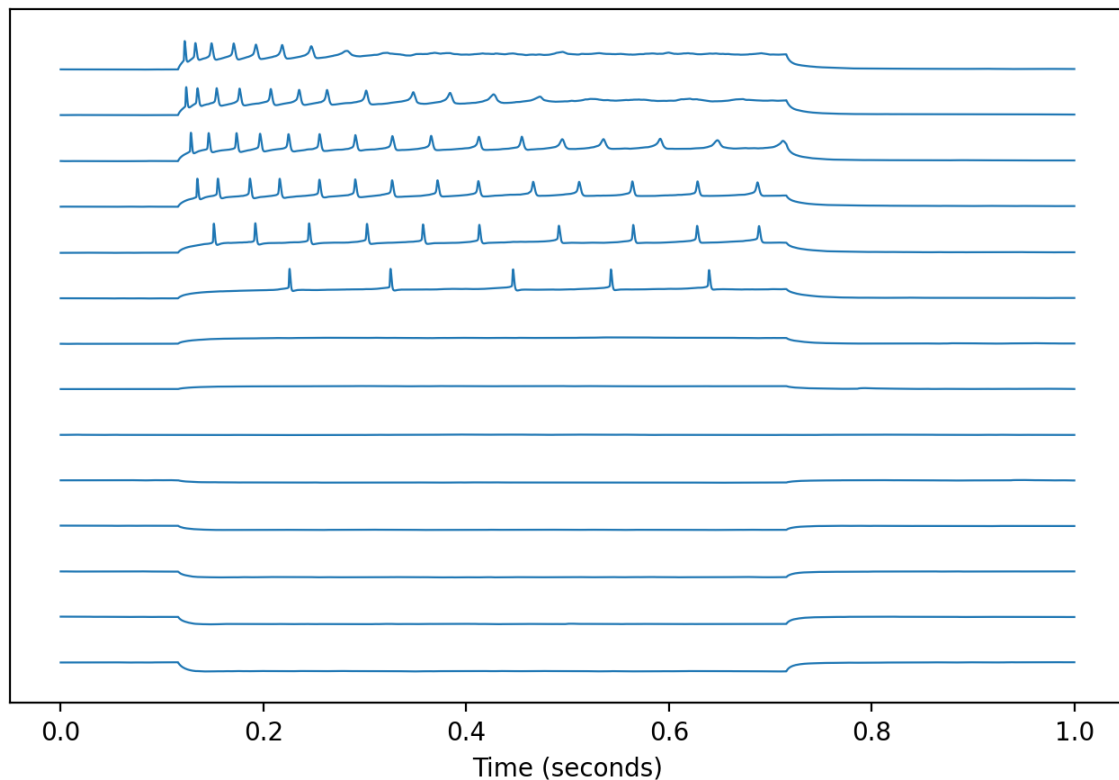


3.4.3 Plotting all sweeps

```
plt.figure(figsize=(8, 5))

# plot every sweep (with vertical offset)
for sweepNumber in abf.sweepList:
    abf.setSweep(sweepNumber)
    offset = 140*sweepNumber
    plt.plot(abf.sweepX, abf.sweepY+offset, color=color_adc, lw=my_lw)

# decorate the plot
plt.gca().get_yaxis().set_visible(False) # hide Y axis
plt.xlabel(abf.sweepLabelX)
plt.show()
```

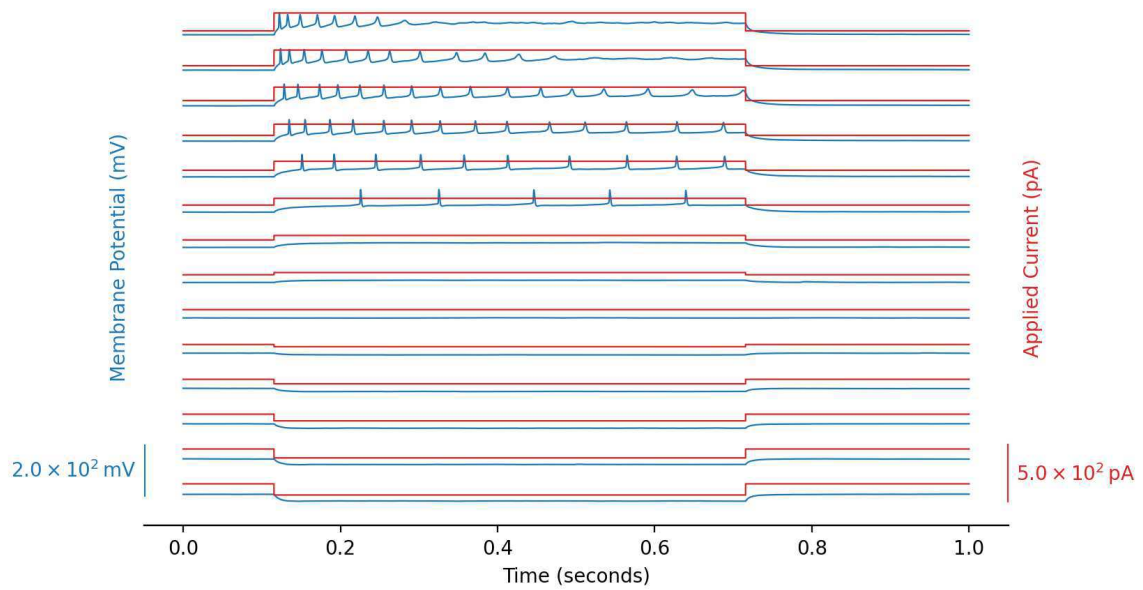


3.4.4 Plotting all sweeps with all inputs

We can improve the previous plot, see the python script `utils/plots.py`:

```
from utils.plots import plot_abf_traces_with_scalebar

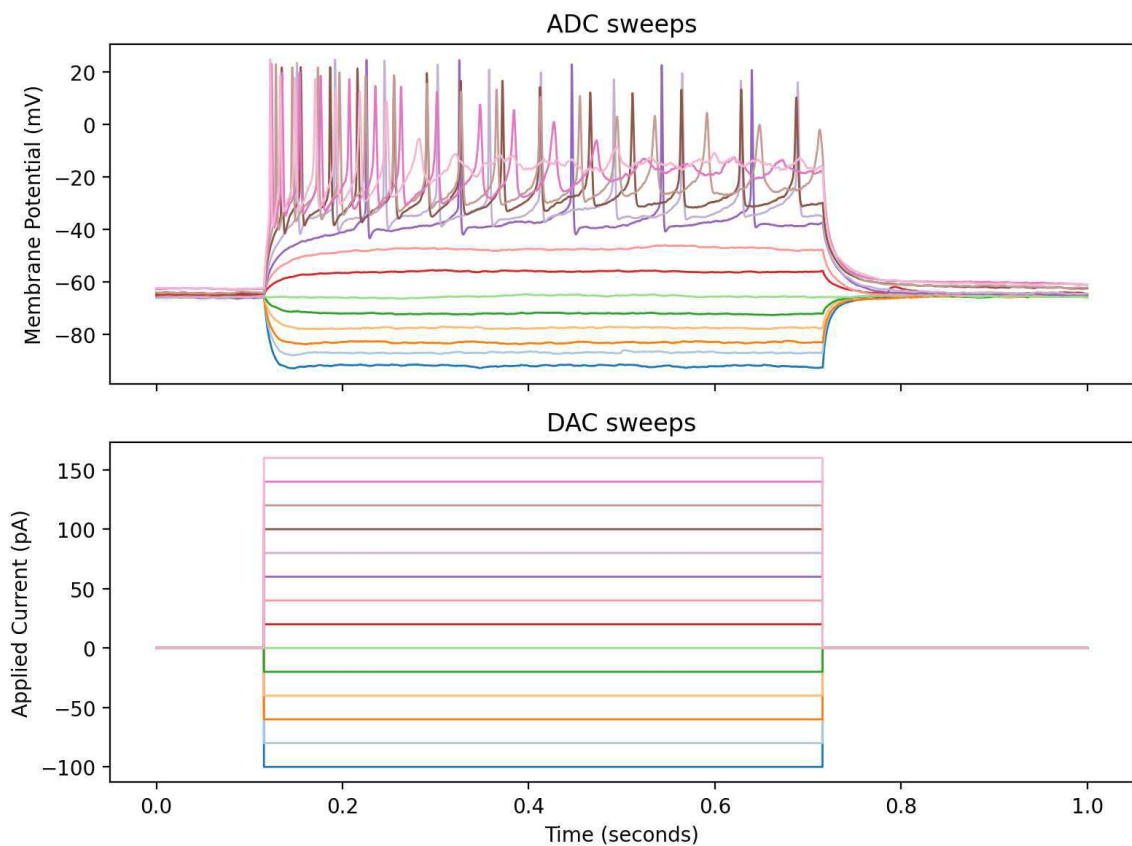
fig, ax1, ax2 = plot_abf_traces_with_scalebar(abf)
plt.show()
```



Another possibility:

```
from utils.plots import plot_abf_sweeps_colored

fig, (ax1, ax2) = plot_abf_sweeps_colored(abf, cmap="tab20", lw=1.0)
plt.show()
```



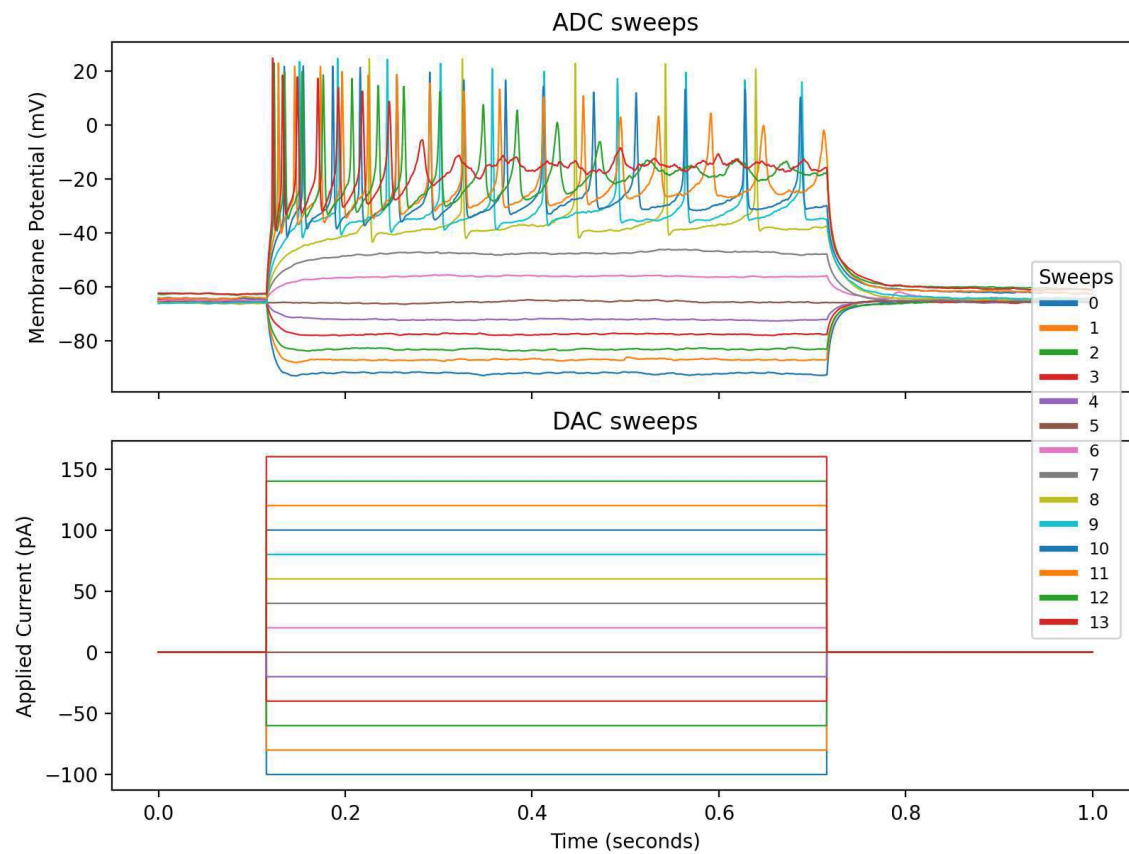
```
from utils.plots import plot_abf_sweeps_with_legend

fig, (ax1, ax2) = plot_abf_sweeps_with_legend(
```

(continues on next page)

(continued from previous page)

```
abf, figsize=(8,6), legend_loc='upper left', legend_bbox=(0.9, 0.7), legend_
pad=0
)
plt.show()
```



Part II

Appendices

INSTALLING PYTHON AND SOME TOOLS

4.1 Main Python distributions for data sciences

When starting with Python for data science, it's important to know the main distributions you can use. These distributions include Python itself, plus tools to manage packages and environments. Here's an overview that works across macOS, Linux, and Windows.

4.1.1 System Python

- Many operating systems come with **Python pre-installed**:
 - macOS and most Linux distributions include Python.
 - Windows does not come with Python pre-installed (you need to download it from [Python.org](https://python.org)).
- Usually an older version (e.g., Python 3.8 or 3.9).
- Good for simple scripts, but installing additional packages may conflict with system tools.

4.1.2 Official Python from Python.org

- The official Python distribution is available at python.org.
- Works on **macOS, Linux, and Windows**.
- You can manually install any additional data science packages (e.g., `numpy`, `pandas`, `matplotlib`) using `pip`.
- Lightweight and cross-platform, but you need to manage dependencies yourself.

4.1.3 Anaconda

- A **full Python distribution for scientific computing and data science**.
- Includes:
 - Python itself
 - Hundreds of pre-installed libraries (`numpy`, `pandas`, `matplotlib`, `scipy`, etc.)
 - Jupyter Notebook / JupyterLab
- Works on **macOS, Linux, and Windows**.
- Large download (~3 GB), but everything is ready-to-use.
- Good choice if you want a complete environment for data science without installing each library manually.

4.1.4 Miniconda

- A **minimal version of Anaconda**, including only Python + the `conda` package manager.
- You install only the packages you need.
- Works on **macOS, Linux, and Windows**.
- Lightweight, flexible, and suitable for reproducible environments.
- Often preferred for creating isolated Python environments per project.

4.1.5 Platform-Specific Package Managers (optional)

- **macOS:** Homebrew can install Python (`brew install python@3.13`).
- **Linux:** System package managers like `apt` (Debian/Ubuntu) or `dnf/yum` (Fedora/CentOS) can install Python.
- **Windows:** Chocolatey can install Python (`choco install python`) if you prefer command-line installation.

⚠ These install **system-wide Python**, not isolated environments, so careful with package conflicts.

📌 Mac users: Homebrew is a great tool to install system software on macOS, but it's generally **not recommended to use brew-installed Python for data science projects**. Why? Because brew installs Python system-wide, which can **conflict with project-specific environments** like `conda` or `venv`. For isolated, reproducible Python environments, prefer **Miniconda or Anaconda** instead.

4.1.6 Summary Table

Distribution	Platforms	Main Feature	Notes
System Python	macOS/Linux	Pre-installed	Might be old; not isolated
Python.org	macOS/Linux/Windows	Official Python	Lightweight; manual package management
Anaconda	macOS/Linux/Windows	Full scientific stack	Large; ready-to-use
Miniconda	macOS/Linux/Windows	Minimal + conda	Lightweight; flexible
Homebrew / apt / dnf / Chocolatey	macOS/Linux/Windows	System package manager	Installs Python and other software system-wide; not isolated

This gives you a clear overview of the **main Python distributions you can use for data science**, regardless of your operating system. Installation instructions and environment setup can be covered later.

📌 Important

⇒ We will therefore focus on the Anaconda solution

4.2 Installing Anaconda and Miniconda

4.2.1 Installing Anaconda

macOS: Go to the [Anaconda Downloads](#) page, download the macOS installer (Graphical or command-line), open the .pkg file, and follow the instructions. Open a terminal and verify the installation:

```
conda --version
```

Linux: Download the Linux installer from [Anaconda Downloads](#). Open a terminal and run:

```
bash ~/Downloads/Anaconda3-<version>-Linux-x86_64.sh
```

Follow the prompts to complete the installation. Verify with:

```
conda --version
```

Windows: Download the Windows installer from [Anaconda Downloads](#). Run the .exe file and follow the instructions. Open **Anaconda Prompt** or **PowerShell** and verify:

```
conda --version
```

4.2.2 Installing Miniconda

macOS: Go to the [Miniconda Downloads](#) page, download the macOS installer, open the .pkg file, and follow the instructions. Open a terminal and verify:

```
conda --version
```

Linux: Download the Linux installer from [Miniconda Downloads](#). Open a terminal and run:

```
bash ~/Downloads/Miniconda3-latest-Linux-x86_64.sh
```

Follow the prompts to complete the installation. Verify with:

```
conda --version
```

Windows: Download the Windows installer from [Miniconda Downloads](#). Run the .exe file and follow the instructions. Open **Anaconda Prompt** or **PowerShell** and verify:

```
conda --version
```

4.2.3 Tips and Notes

- Optionally add conda to your PATH during installation to use it from any terminal.
- Update conda after installation:

```
conda update conda
```

- Miniconda is recommended for a lightweight setup.
- Usage of conda environments and package installation will be covered in later sections.

4.2.4 Summary Table

Step	macOS	Linux	Windows
Download installer	Anaconda / Mini-conda	Same	Same
Run installer	.pkg	bash ~/Downloads/installer.sh	.exe
Verify installation	conda --version	conda --version	conda --version
Notes	Graphical or CLI installer	Terminal-based	Use Anaconda Prompt or PowerShell

4.3 Conda

Conda is a package manager for Python and other languages. It helps you install packages and manage dependencies easily.

4.3.1 Basics

```
# check that it is correctly installed:
conda --version

# keep Conda up-to-date with:
conda update conda

# install a package (Replace `numpy` with the desired package name):
conda install numpy

# install a specific version:
conda install numpy=1.25

# install multiple packages at once:
conda install numpy pandas matplotlib

# updating packages:
conda update numpy

# removing packages:
conda remove numpy

# searching for packages(replace `package_name` with the
# name of the package you want to find):
conda search package_name
```

4.3.2 Conda tips

Update Conda regularly to get bug fixes and security updates.

If a package is not found, check alternative channels:

```
conda install -c conda-forge package_name
```

4.3.3 Summary of common Conda commands

Task	Command
Check Conda version	<code>conda --version</code>
Update Conda	<code>conda update conda</code>
Install package	<code>conda install package_name</code>
Install specific version	<code>conda install package_name=version</code>
Install multiple packages	<code>conda install package1 package2</code>
Update package	<code>conda update package_name</code>
Remove package	<code>conda remove package_name</code>
Search for package	<code>conda search package_name</code>
Install from channel	<code>conda install -c conda-forge package_name</code>

4.4 Conda virtual environment

4.4.1 Using Virtual Environments

Why Use a Virtual Environment in Python?

The problem without a virtual environment:

- By default, when you install a library with `pip install`, it goes into the **system-wide Python**.
- Risks:
 - ⚠ Version conflicts between projects (e.g., one project needs `numpy==1.20`, another `numpy==1.26`).
 - ⚠ Risk of breaking system tools that rely on Python (macOS and Homebrew depend on it).
 - ⚠ Environment quickly polluted with dozens of unnecessary packages.

Solution: Virtual Environments

A virtual environment = an **isolated copy of Python** with its own libraries.

Advantages:

- Project-by-project isolation.
- No conflicts between library versions.
- Easier to share and reproduce a project (`requirements.txt` or `environment.yml`).
- You can delete a project without polluting the system.

Two Main Choices: `venv` vs `conda`

`venv` (native Python virtual environments)

- Included in Python (`python -m venv myenv`).
- Lightweight, simple to use.
- Package management via `pip install`.
- Good for:
 - Lightweight projects (Flask, Django, scripts).
 - General development.

⚠ Limitations:

- `pip` installs only Python libraries.
- Some heavy libraries (numpy, scipy, torch, tensorflow...) may require compilation → possible errors.

`conda` (Anaconda/Miniconda environments)

- Also creates isolated environments (`conda create -n myenv python=3.10`).
- Can install not only Python libraries, but also **system dependencies** (BLAS, MKL, CUDA, etc.).
- Precompiled package distribution → fast and reliable installation.
- Good for:
 - Data science and machine learning (numpy, pandas, scikit-learn, PyTorch, TensorFlow).
 - Multi-language projects (Python + R + CUDA...).

⚠ Limitations:

- Heavier than `venv`.
- Package management can be slightly slower at times.

🔔 Important

⇒ Another reason to focus on the Anaconda solution

4.4.2 Conda Virtual Environments

Conda: Creating a New Environment

```
# create a new Conda environment with a specific Python version (Replace `myenv`  
# with the name of your environment and `3.11` with the desired Python version)  
conda create --name myenv python=3.11  
  
# Activate the environment before working in it:  
conda activate myenv  
  
# When you are done, deactivate the environment to return to the base environment:  
conda deactivate
```


Conda: Listing and Removing Environments

```
# list all available environments:
conda env list

# remove an environment completely:
conda remove --name myenv --all
```

Conda: Installing Packages in an Environment

```
# install a package in the active environment:
conda install numpy

# install a specific version of a package:
conda install numpy=1.25

# install multiple packages at once:
conda install numpy pandas matplotlib
```

Conda: Updating and Removing Packages

```
# update a package in the current environment:
conda update numpy

# remove a package from the environment:
conda remove numpy
```

Conda: Exporting and Reproducing Environments

```
# to share or reproduce an environment, export it to a YAML file:
conda env export > environment.yml

# create an environment from a YAML file:
conda env create -f environment.yml
```

4.4.3 Conda Tips

- Always use separate environments for different projects to avoid conflicts.
- Update Conda regularly with `conda update conda`.
- Use the `conda-forge` channel if a package is not found in the default channels:

```
conda install -c conda-forge package_name
```

Summary Table of Common Conda Environment Commands

Task	Command
Create environment	<code>conda create --name myenv python=3.11</code>
Activate environment	<code>conda activate myenv</code>
Deactivate environment	<code>conda deactivate</code>
List environments	<code>conda env list</code>
Remove environment	<code>conda remove --name myenv --all</code>
Install package	<code>conda install package_name</code>
Install specific version	<code>conda install package_name=version</code>
Install multiple packages	<code>conda install package1 package2</code>
Update package	<code>conda update package_name</code>
Remove package	<code>conda remove package_name</code>
Export environment	<code>conda env export > environment.yml</code>
Create from file	<code>conda env create -f environment.yml</code>

4.5 Setting Up the Conda Virtual Environment for This Project

This section is intended for macOS users.

To run this Jupyter Book, I make use of a conda virtual environment, whose recipe i contained in the file `environment.yml` that describes everything needed to create the conda environment named `python-dsspikes-env`:

```
!cat environment.yml
```

```
name: python-dsspikes-env
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.11
  - numpy=1.26 # compatible version for macOS
  - matplotlib=3.8
  - seaborn=0.12
  - jupyter-book=1.0.4
  - jupyterlab=4.1
  - ipykernel
  - pip
  - pip:
    - pyabf==2.3.8 # install via pip
```

`name: python-dsspikes-env` tells conda what name you assign to the environment. When you run:

```
conda env create -f environment.yml
```

Conda reads that line and creates an environment with that name. So after creation, you'll activate it with:

```
conda activate python-dsspikes-env
```

Each time you run it, conda will move one step “up”: If you're inside `python-dsspikes-env`, it will go back to `(base)`. If you're already in `(base)`, it will deactivate completely (no environment active). So the cycle is:

```
conda activate python-dsspikes-env
# ... work here ...
conda deactivate
```

The quickest way to check which conda environment is active is:

```
conda info --envs
```

or its shorthand:

```
conda env list
```

hence:

```
(python-dsspikes-env) data-science-spikes$ conda env list
# conda environments:
#
base                  /Users/campillo/miniforge3
myenv                 /Users/campillo/miniforge3/envs/myenv
python-dsspikes-env  * /Users/campillo/miniforge3/envs/python-dsspikes-env
```

- The * shows which environment is currently active.
- In your shell prompt, the active environment name also appears in parentheses, e.g. (python-dsspikes-env) data-science-spikes\$ (here (conda_virtual_env) directory_name).

To check the active conda environment inside a Jupyter notebook, you have a few options:

1. Check sys.executable

```
import sys
sys.executable # This shows the path to the Python binary being used.
```

```
'/opt/anaconda3/envs/python-dsspikes-env/bin/python'
```

2. Check environment variables

```
import os
os.environ.get("CONDA_DEFAULT_ENV")
```

```
'python-dsspikes-env'
```

3. Print Python packages & versions To confirm everything is coming from the right env:

```
!which python
!python --version
!pip list | grep -E "pyabf|matplotlib|seaborn"
```

```
/opt/anaconda3/envs/python-dsspikes-env/bin/python
```

```
Python 3.11.13
```

```
matplotlib          3.8.4
matplotlib-inline    0.1.7
pyabf                2.3.8
seaborn              0.12.2
```

and !pip list for the complete list.

Setting Up the Conda Virtual Environment for This Project

This project uses Python packages such as numpy, matplotlib, seaborn, and pyabf.

To ensure reproducibility and avoid conflicts with other Python projects, we recommend using a **dedicated Conda virtual environment**.

1. Create the environment - Run the following command in your terminal:

```
conda env create -f environment.yml
```

This will create a new environment named `jupyter-env` (as specified in `environment.yml`). All required packages for this Jupyter Book project will be installed.

2. Activate the environment

```
conda activate jupyter-env
```

Your terminal prompt should now show `(jupyter-env)`, indicating the environment is active.

3. Make the environment available in Jupyter

```
python -m ipykernel install --user --name=jupyter-env --display-name "Python_↵  
↵(jupyter-env)"
```

This allows notebooks to select the correct kernel.

4. Verify installation - You can test that everything is installed correctly:

```
python -c "import numpy, matplotlib, seaborn, pyabf; print('All imports OK!')"
```

If no errors appear, the environment is ready to use.

5. Launch Jupyter Lab or Notebook

```
# Launch Jupyter Lab  
jupyter lab  
  
# or launch classic Jupyter Notebook  
jupyter notebook
```

In the notebook (top right), select Kernel → Python (jupyter-env).



6. Updating the environment – If you modify `environment.yml` later (e.g., adding packages), update the environment:

```
conda env update -f environment.yml --prune
```

`--prune` removes packages no longer listed in `environment.yml`.

Notes:

- Keep all project-specific packages inside the virtual environment; do not install them in base.
- For reproducibility, commit `environment.yml` to your repository.

INTERACTIVE COMPUTING WITH JUPYTER ECOSYSTEM

Jupyter, Jupyter Notebook, JupyterLab, Binder / MyBinder, Jupyter{book}, Colab, Deepnote

5.1 Jupyter and around

The Jupyter ecosystem evolved to make computing **interactive, reproducible, and shareable**. It began with **IPython**, an enhanced Python shell for rapid experimentation, and expanded into **Jupyter** to support multiple languages through a decoupled kernel-interface model. **Jupyter Notebook** introduced a web-based environment combining code, outputs, and narrative text, ideal for teaching, research, and data analysis. **JupyterLab** provides a modern, flexible workspace for managing notebooks, scripts, and data in complex projects. Finally, **Jupyter{book}** allows structured, publication-quality books and websites to be built from notebooks and Markdown, fully executable and shareable. Together, these tools address the need for **interactive coding, reproducibility, multi-language support, and clear communication of computational results**, even beyond Python:

- **Jupyter** is an open-source project that evolved from **IPython**. IPython originally provided an enhanced interactive Python shell, with powerful features like introspection, rich media, and tab-completion, making Python more user-friendly for experimentation and data analysis. Jupyter extended this idea to **multiple programming languages** (Python, R, Julia, and more) by separating the **kernel** (which executes code) from the **interface** (which displays results interactively).
- **Jupyter Notebook** builds on this foundation, providing a web-based interactive environment where users can write and execute code, display rich outputs (plots, images, LaTeX, widgets), and combine them with narrative text. This allows notebooks to serve as **reproducible documents** for data analysis, teaching, and scientific communication.
- **JupyterLab** is the next-generation interface for Jupyter, offering a **modular, flexible environment** where users can work with notebooks, text editors, terminals, and data files all in one workspace. It improves productivity for complex projects, supports extensions, and makes multi-document workflows smoother than classic notebooks.
- **Binder/MyBinder** is a cloud service that allows users to **launch fully executable Jupyter environments** directly from GitHub repositories. It lets anyone run notebooks or Jupyter Books without installing anything locally, making content fully interactive and reproducible online.
- **Jupyter{book}** extends the Jupyter ecosystem by allowing users to turn **collections of notebooks and Markdown files into interactive, publication-quality books and websites**. Unlike standalone notebooks, Jupyter Books provide structured chapters, table-of-contents, cross-references, and can be executed to show live outputs, making them ideal for teaching, tutorials, and reproducible scientific publications.

```
Jupyter Ecosystem Evolution (approximate years)
=====
IPython (2001) - interactive Python shell
|
▼
Jupyter (2014) - multi-language kernel architecture
|
```

(continues on next page)

(continued from previous page)

- └─ Jupyter Notebook (2015) – web-based interactive notebooks
 - web-based interactive notebooks
- └─ JupyterLab (2018)
 - modern IDE-like workspace for notebooks & files
- └─ Binder / MyBinder (2017)
 - cloud service for executable notebooks (and later books)
- └─ Jupyter{book} (2018+)
 - structured, publication-quality books from notebooks & Markdown

5.2 Colab

Colab is Google's cloud version of Jupyter Notebook, fully integrated with the Jupyter ecosystem, making it easy to run, share, and collaborate on notebooks online.

Colab (2017) sits in this ecosystem as a **cloud-hosted Jupyter Notebook environment**:

- Runs notebooks **without local setup**.
- Provides **free CPU/GPU/TPU resources**.
- Enables **real-time collaboration** like Google Docs.
- Fully compatible with **.ipynb notebooks**, so you can open notebooks from GitHub or Jupyter Book in Colab.

Colab is designed:

- For **students, researchers, or teams** who don't want to install Python locally.
- To **share interactive notebooks** with reproducible results.
- To **test notebooks from GitHub** quickly.

5.3 Jupyter {book}

jupyter {book} and two important files:

- `_config.xml`
- `_toc.yml`

Launch into interactive computing interfaces

5.3.1 Jupyter backends

the Jupyter inline backend is what converts your Matplotlib figures into embedded images inside notebooks, and `%config InlineBackend...` is how you control their quality and format.

```
%config InlineBackend.figure_format = 'pdf'      # fine but could be combersome
%config InlineBackend.figure_format = 'svg'      #
%config InlineBackend.figure_format = 'retina'   #

%config InlineBackend.figure_format = 'png'
%config InlineBackend.rc = {'figure.dpi': 200}   # or 300 for print quality
```

with `retina` the Jupyter inline backend actually tells Matplotlib to render the figure at double the standard DPI (so if your default is 100 dpi, it makes a 200 dpi PNG). Then the notebook displays it at the normal on-screen size, so it looks sharper on high-density displays (like MacBooks) and also when the image gets embedded in the LaTeX/PDF build.

Other backends:

```
%matplotlib inline      # use inline backend
%matplotlib notebook    # interactive plots inside notebook
%matplotlib widget       # interactive with ipywidgets
```


DIAGNOSTIC

6.1 Myst

This page tests **MyST Markdown extensions**.

6.1.1 Task list

- [x] Done
 - [] Not yet done
-

6.1.2 Admonitions

Note

This is a *note* admonition.

Warning

This is a *warning* admonition.

6.2 Emoji Test Page

This page demonstrates using emojis in a Jupyter Book PDF.

6.2.1 Inline Emoji

Here is a lightbulb emoji inline:  

Here is a rocket emoji inline:  

6.2.2 Emoji with text

- :emoji:1F4A1 **Idea:** Always document your data!
- :emoji:1F680 **Launch:** Start your analysis.
- :emoji:1F680 **Rocket + Math:** $E = mc^2$:emoji:1F680

6.2.3 Emoji in a list

1. :emoji:1F4DA Read the documentation ?
2. :emoji:1F4BB Use Python ?
3. :emoji:1F4A1 Generate ideas ?

6.2.4 Emoji in headers

:emoji:1F680 Getting Started

:emoji:1F4A1 Tips & Tricks